JAVA Platform Module System (JPMS)

Project Jigsaw

Literature

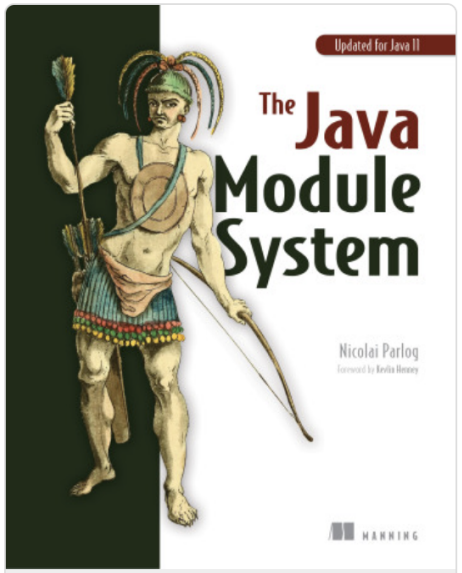
manning.com/books/the-java-module-system

MANNING

search inside this and other books

catalog MEAP liveBook liveVideo liveProject liveAudio free content register pB

Save 35% on your order! Find out how ▸



look inside

Interview with Nicolai Parlog

The Java Module System ♡

| 4.4 ★★★★½ 25 amazon ratings

Nicolai Parlog

Foreword by Kevlin Henney

June 2019 · ISBN 9781617294280 · 440 pages

printed in black & white

Java/JVM

eBook pdf, ePub, online

print includes eBook

subscription from $19.99

Java's much-awaited "Project Jigsaw" is finally here! Java 11 includes a built-in modularity framework, and *The Java Module System* is your guide to discovering it. In this new book, you'll learn how the module system improves reliability and maintainability, and how it can be used to reduce tight coupling of system components.

Introduction

Project Jigsaw was an umbrella project with the new features aimed at two aspects:

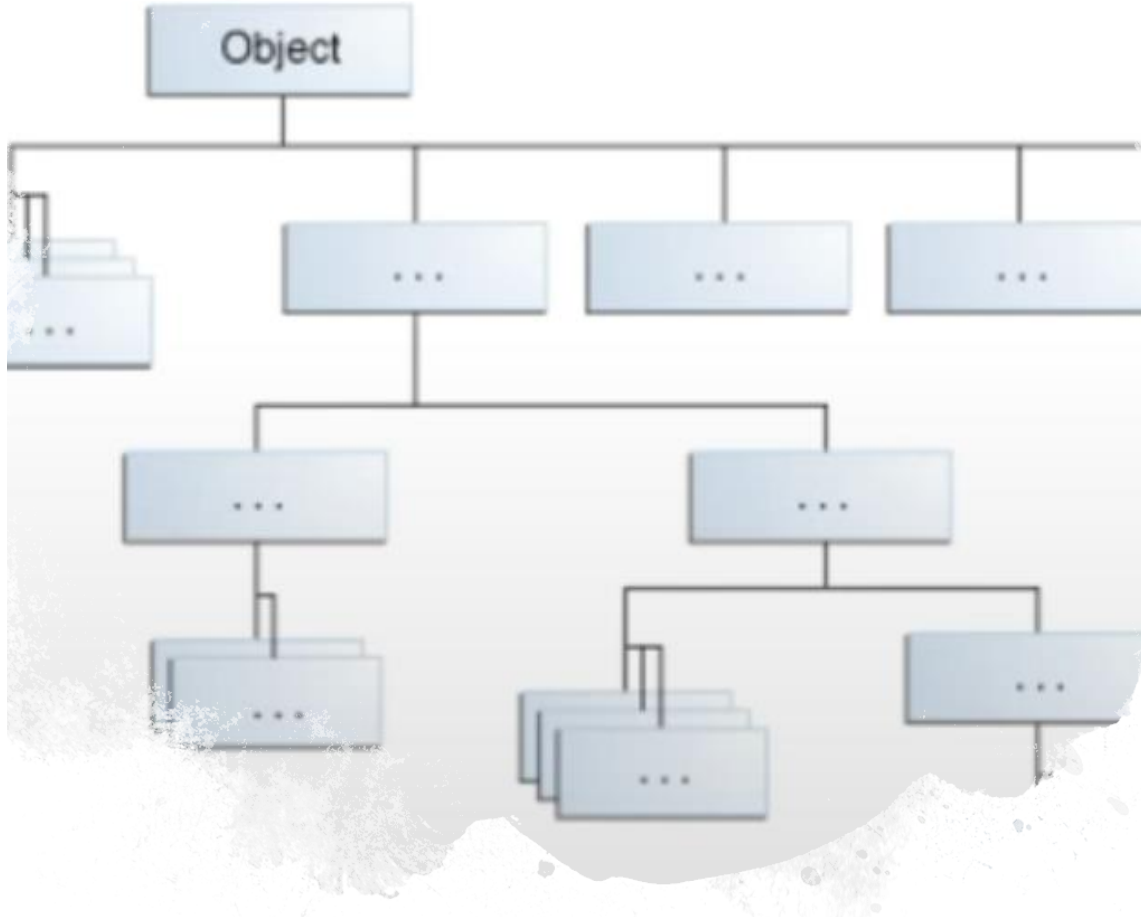
- The introduction of module system in the Java language
- and its implementation in JDK source and Java runtime

Modular Development JDK9

Simply put, modularity is a design principle that helps us to achieve:

- loose coupling between components
- clear contracts and dependencies between components
- hidden implementation using strong encapsulation

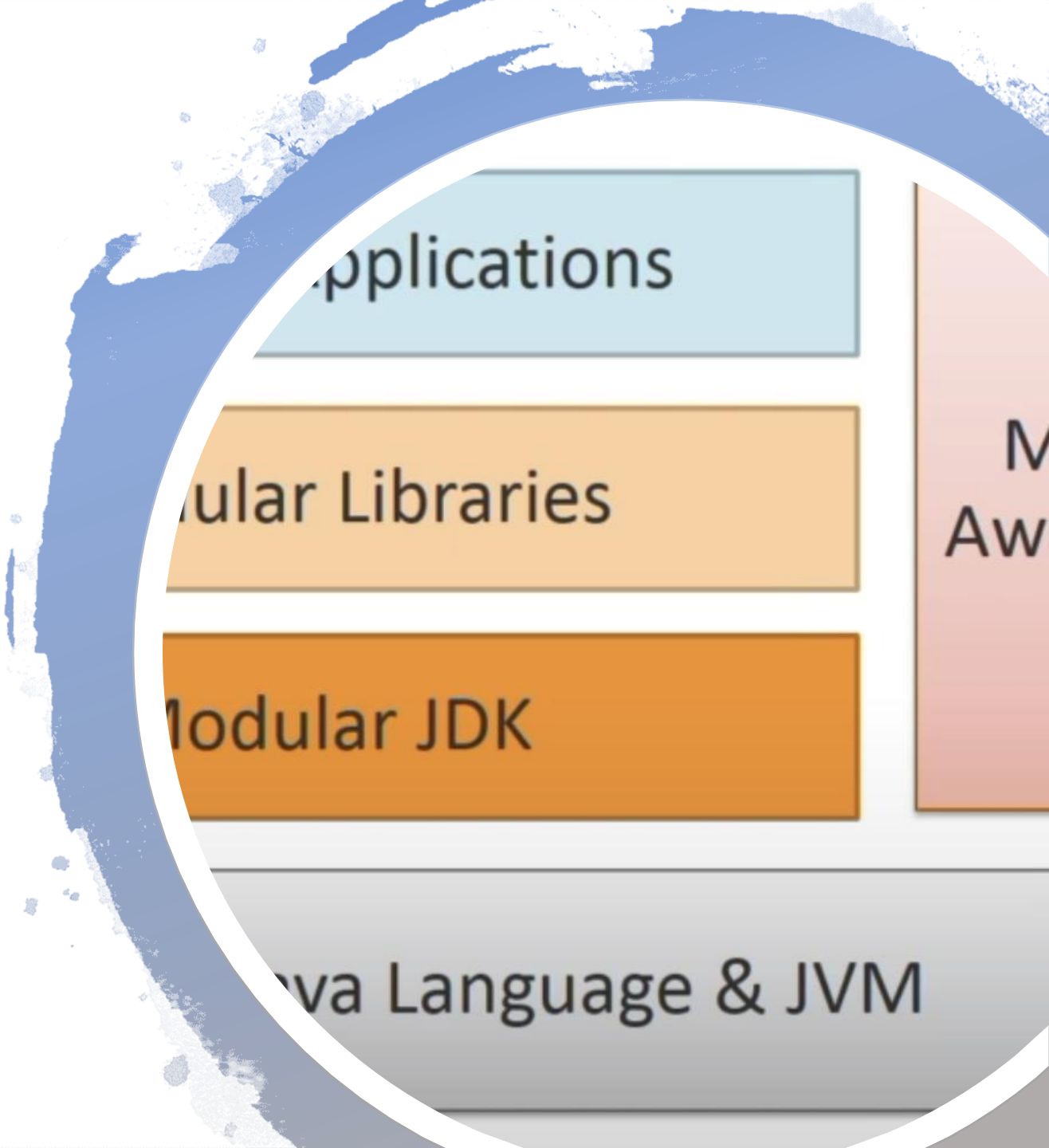
Programs are Classes



Programs are ~~Classes~~ Packages



Programs are
~~Classes~~ ~~Packages~~
Modules



Unit of Modularity

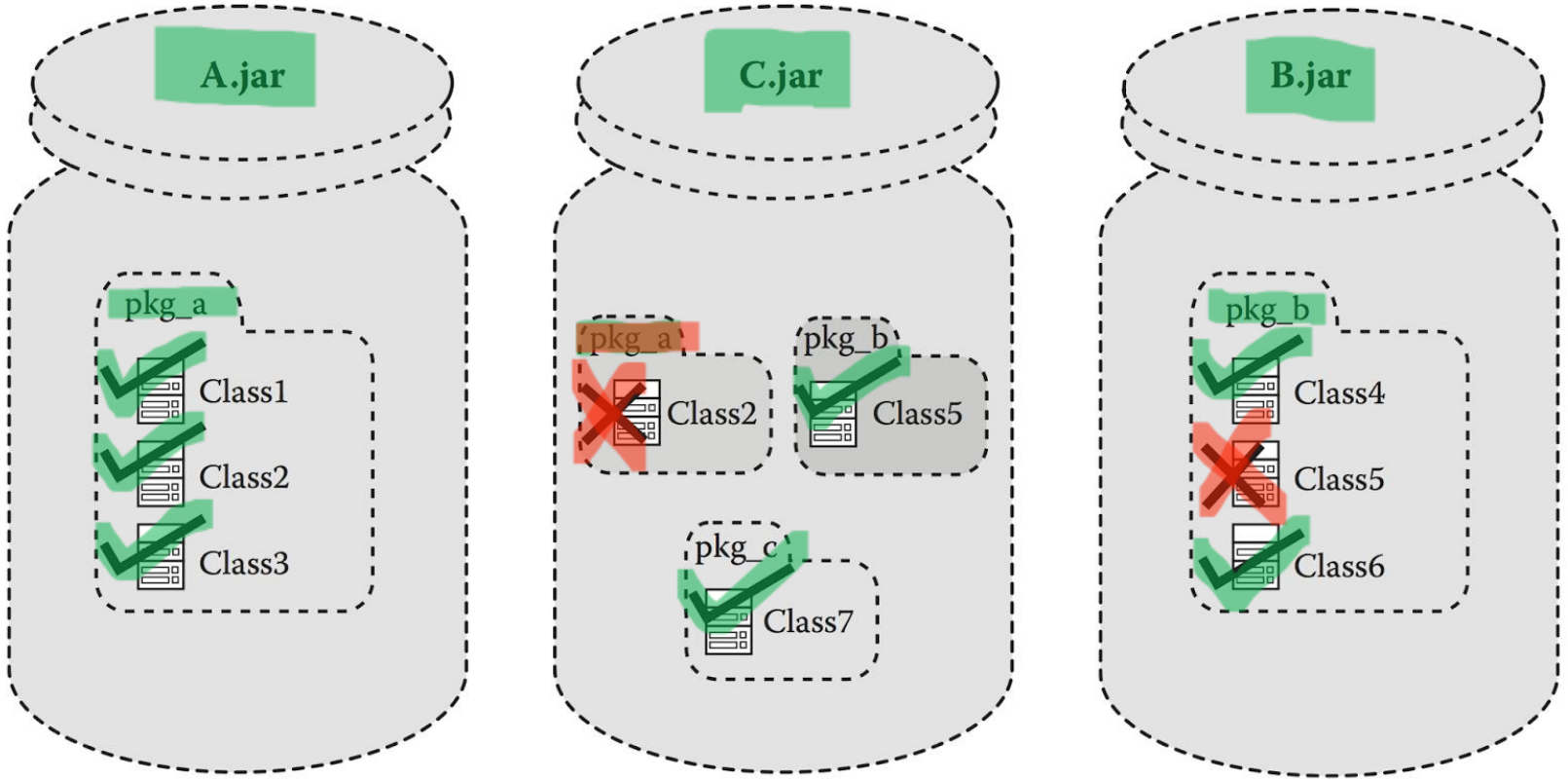
In the Java world, especially with **OSGi**, JARs were considered as the unit of modularity. JARs did help in grouping the related components together, but they do have some limitations:

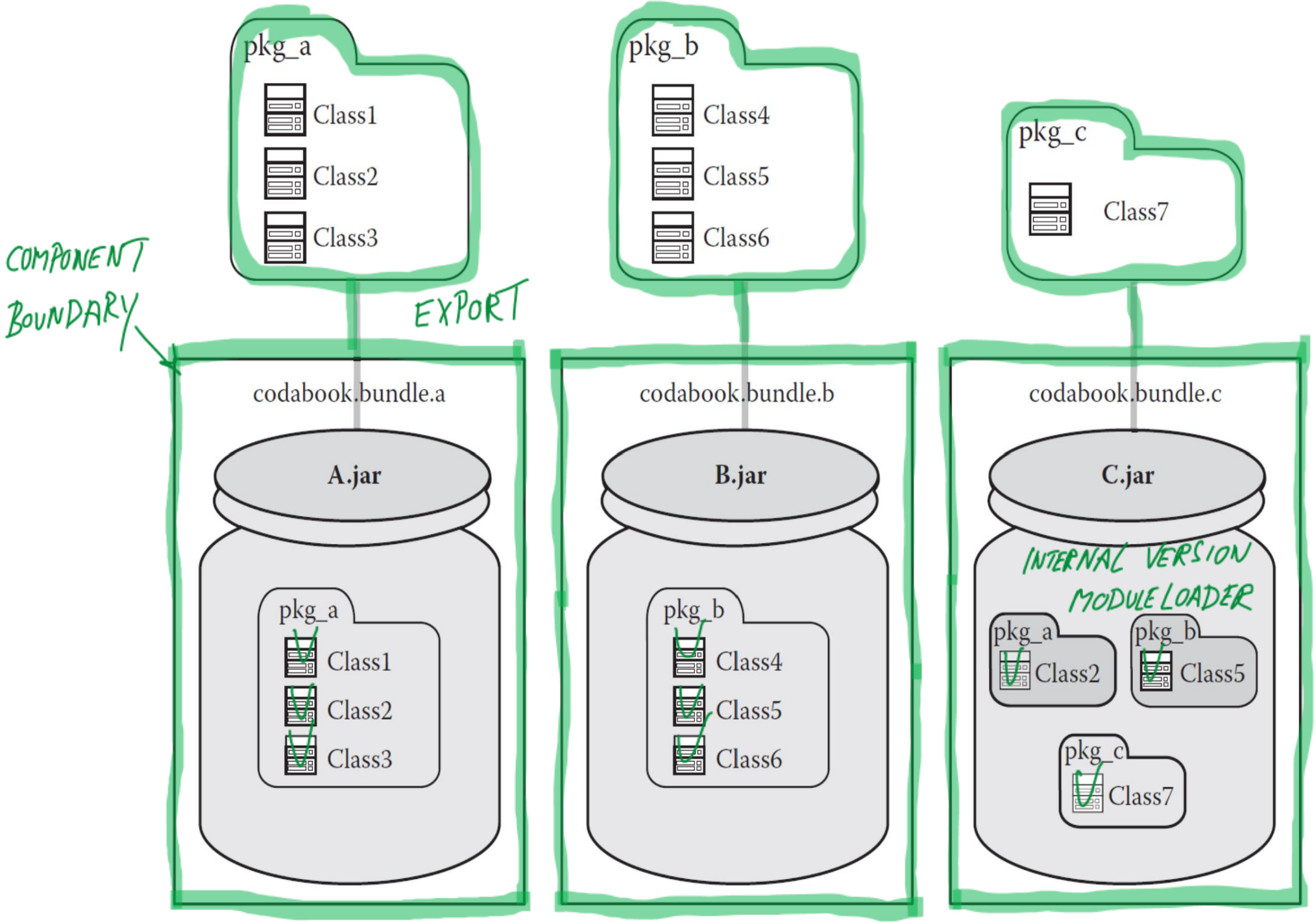
- explicit contracts and dependencies between JARs
- weak encapsulation of elements within the JARs

JAR Hell

- There was another problem with JARs – the JAR hell. Multiple versions of the JARs lying on the classpath, resulted in the *ClassLoader* loading the first found class from the JAR, with very unexpected results.
- The other issue with the JVM using classpath was that the compilation of the application would be successful, but the application will fail at runtime with the *ClassNotFoundException*, due to the missing JARs on the classpath at runtime.

Problem w. Flat Class-Path




pkg_a
Class1
Class2
Class3
pkg_b
Class4
Class5
Class6
pkg_c
Class7
COMPONENT BOUNDARY
EXPORT
codabook.bundle.a
codabook.bundle.b
codabook.bundle.c
A.jar
B.jar
C.jar
pkg_a
Class1
Class2
Class3
pkg_b
Class4
Class5
Class6
INTERNAL VERSION MODULE LOADER
pkg_a
Class2
pkg_b
Class5
pkg_c
Class7

New Unit of Modularity

With all these limitations, when using JAR as the unit of modularity, the Java language creators came up with a new construct in the language called modules.

And with this, there was a whole new modular system planned for Java.

Project Jigsaw

The primary motivations for this project are:

- **create a module system for the language** – implemented under **JEP 261**
- **apply it to the JDK source** – implemented under **JEP 201**
- **modularize the JDK libraries** – implemented under **JEP 200**
- **update the runtime to support modularity** – implemented under **JEP 220**
- **be able to create smaller runtime with a subset of modules from JDK** – implemented under **JEP 282**

Another important initiative is to **encapsulate the internal APIs** in the JDK, those who are under the *sun.** packages and other non-standard APIs. These APIs were never meant to be used by public and were never planned to be maintained. But the power of these APIs made the Java developers leverage them in the development of different libraries, frameworks, and tools.

New Tools for Modularity

- **jdeps** – helps in analyzing the code base to identify the dependencies on JDK APIs and the third party JARs. It also mentions the name of the module where the JDK API can be found. This makes it easier in modularizing the code base
- **jdeprscan** – helps in analyzing the code base for usage of any deprecated APIs
- **jlink** – helps in creating a smaller runtime by combining the application's and the JDK's modules
- **jmod** – helps in working with jmod files. jmod is a new format for packaging the modules. This format allows including native code, configuration files, and other data that do not fit into JAR files

Module System Architecture

A module definition file, named as *module-info.java*, contains:

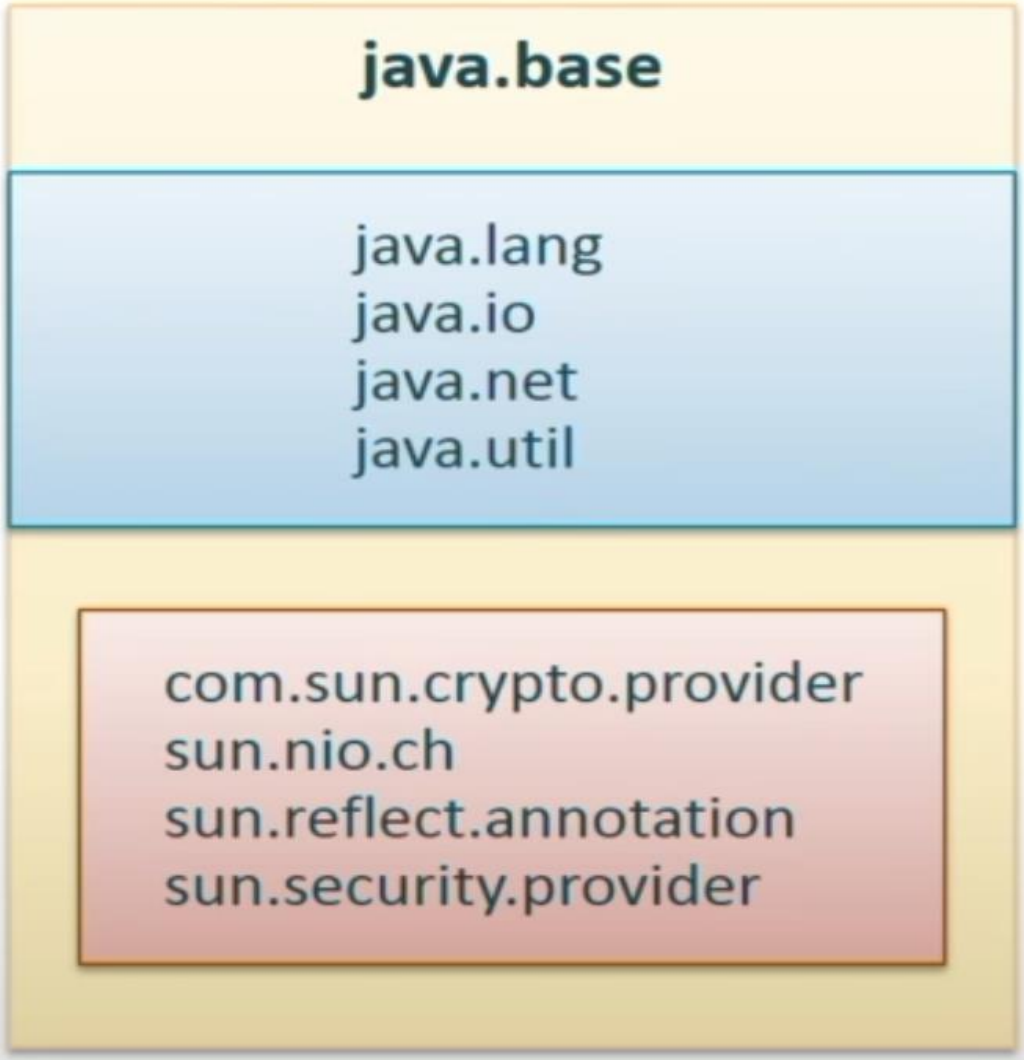
- its name
- the packages it makes available publicly
- the modules it depends on
- any services it consumes
- any implementation for the service it provides

The last two items in the above list are not commonly used. They are used only when services are being provided and consumed via the *java.util.ServiceLoader* interface.

Module Definition Terminologies

- ***module***: the module definition file starts with this keyword followed by its name and definition
- ***requires***: is used to indicate the modules it depends on; a module name has to be specified after this keyword
- ***transitive***: is specified after the *requires* keyword; this means that any module that depends on the module defining *requires transitive <modulename>* gets an implicit dependence on the <*modulename*>
- ***exports***: is used to indicate the packages within the module available publicly; a package name has to be specified after this keyword
- ***opens***: is used to indicate the packages that are accessible only at runtime and also available for introspection via Reflection APIs; this is quite significant to libraries like Spring and Hibernate, highly rely on Reflection APIs; *opens* can also be used at the module level in which case the entire module is accessible at runtime
- ***uses***: is used to indicate the service interface that this module is using; a type name, i.e., complete class/interface name, has to specified after this keyword
- ***provides … with …***: they are used to indicate that it provides implementations, identified after the *with* keyword, for the service interface identified after the *provides* keyword

java.base




```
// module-info.java
module java.base {
  exports java.lang;
  exports java.io;
  exports java.net;
  exports java.util;
}
```

Accessibility

Accessibility (JDK 1 – JDK 8)

- public
- protected
- package
- private

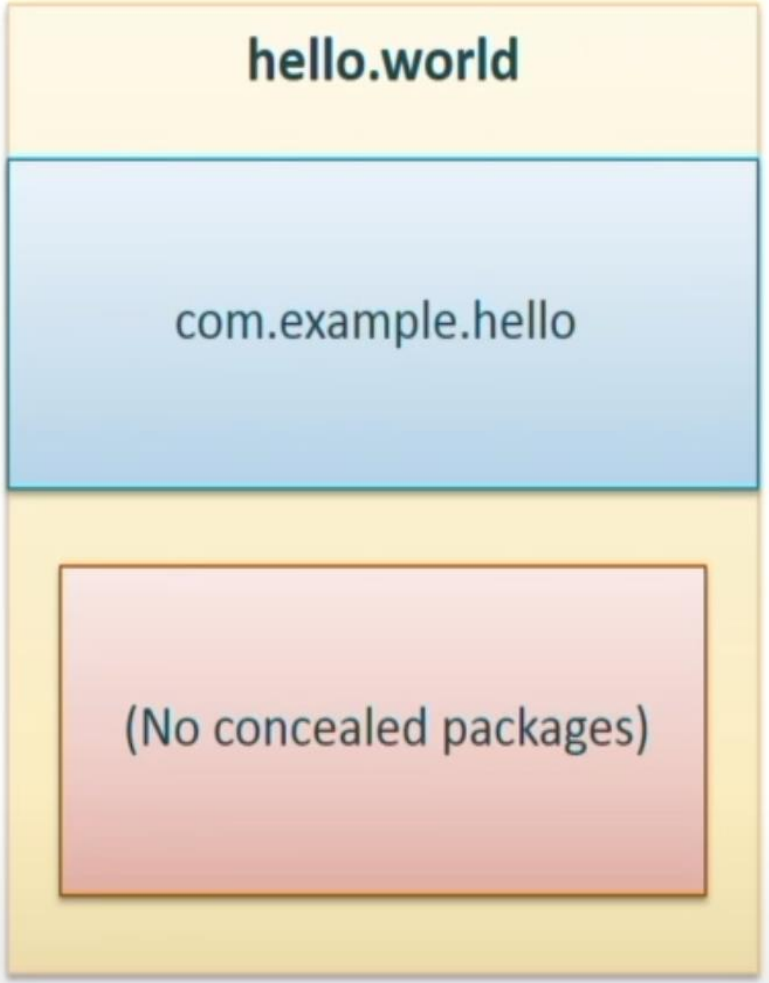
Accessibility (JDK 9)

- *public to everyone*
- *public but only to friend modules*
- *public only within a module*
- protected
- package
- private

“Hello world” as a Module

```
// src/com/example/hello/SayHello.java
package com.example.hello;
import java.lang.*;
public class SayHello {
  public static void main(String[] args) {
    System.out.println(“Hello world”);
  }
}
```

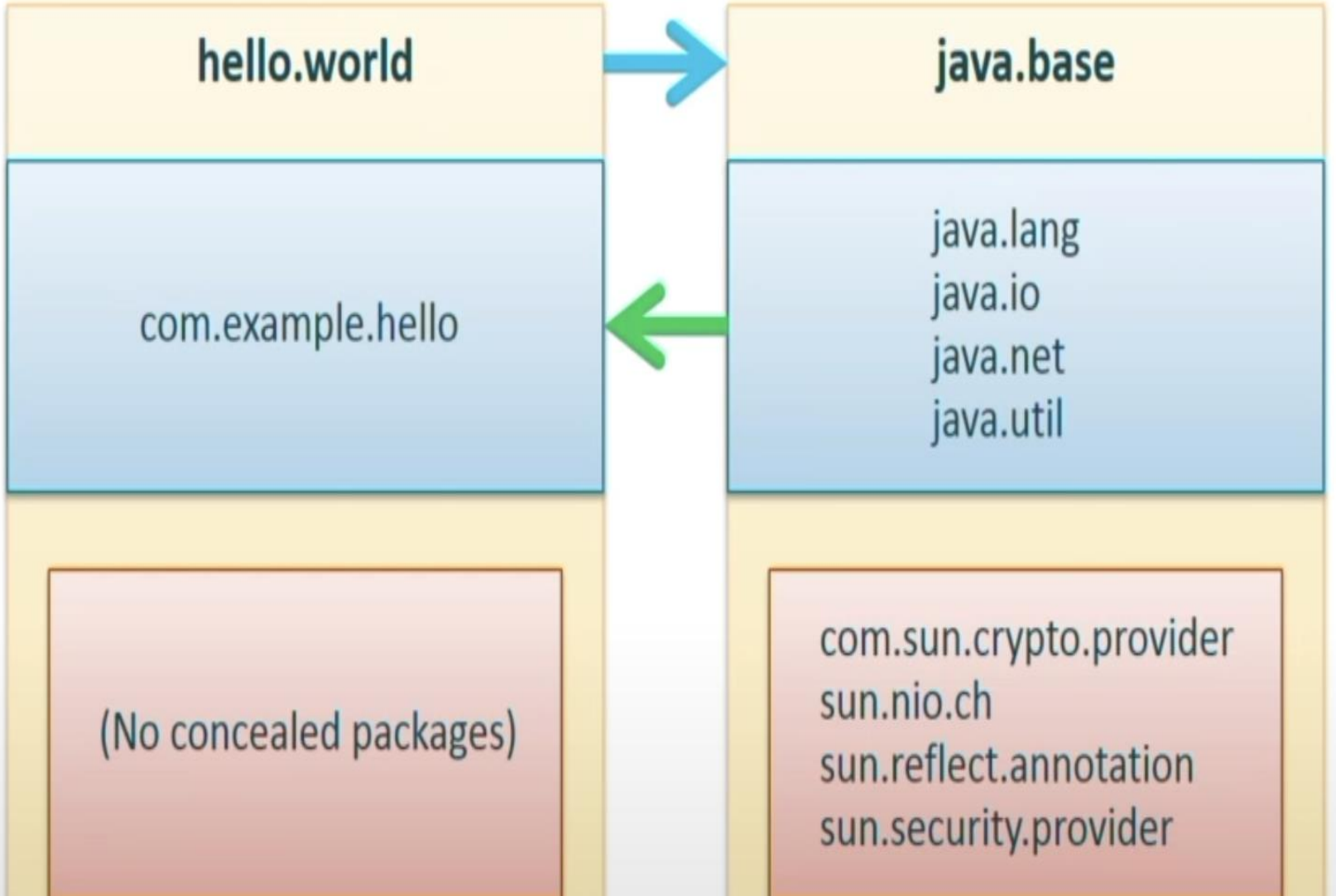
```
// src/module-info.java
module hello.world {
  exports com.example.hello;
}
```



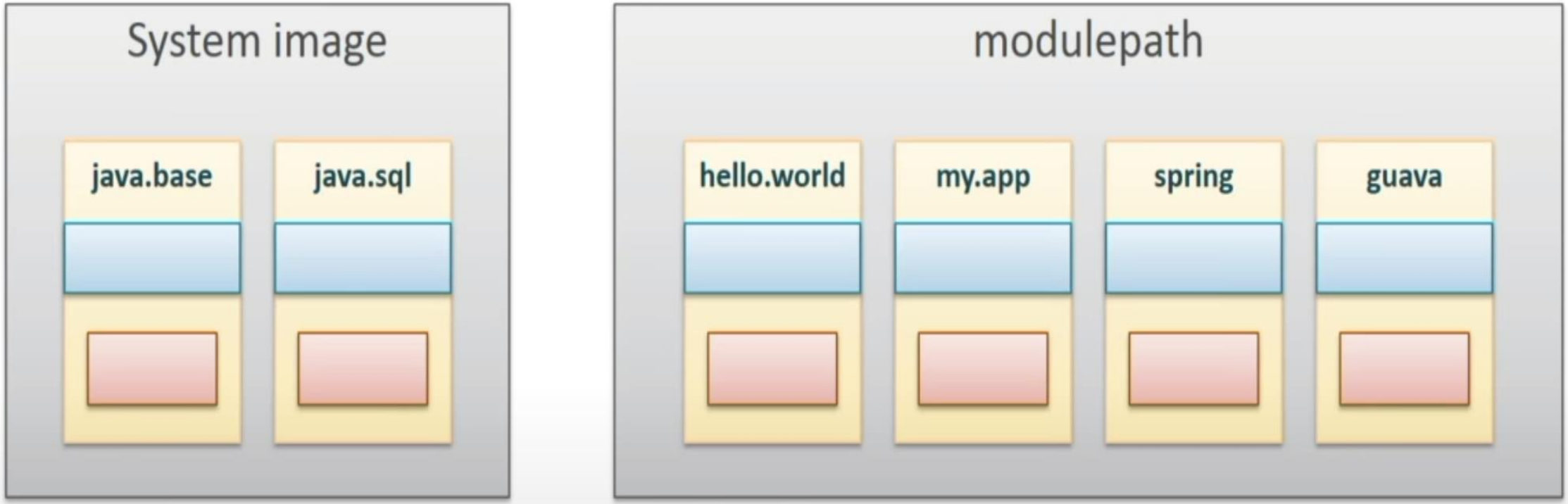

```
javac -d classes -sourcepath src
      module-info.java
      com/example/hello/SayHello.java
```

Reusing a Module

```
// src/module-info.java
module hello.world {
  exports  com.example.hello;
  requires java.base;
}
```



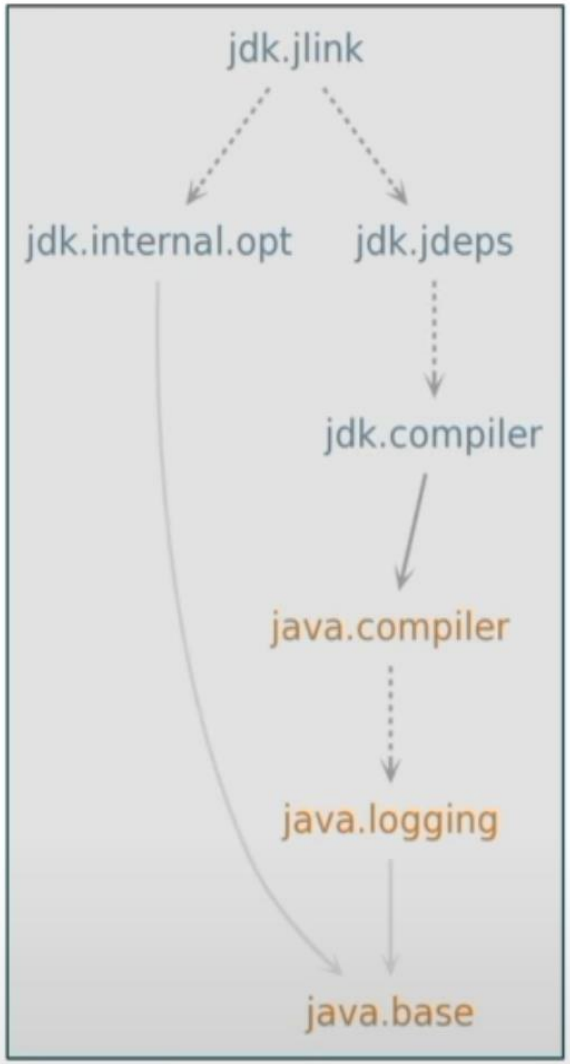
Running a Modular App




```
java -p mods -m hello.world
```

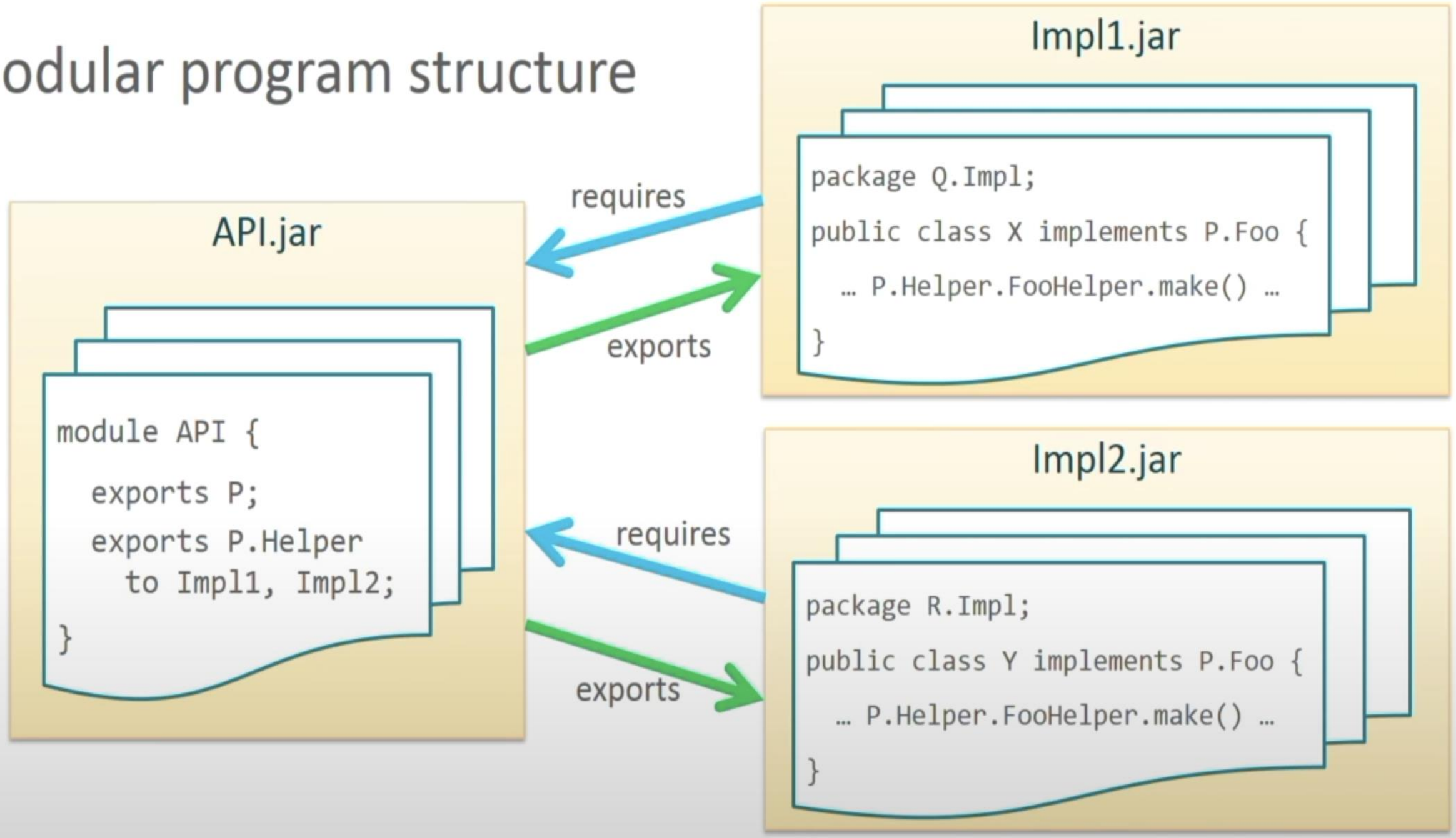
Running a Modular Application

- No missing dependencies
- No cyclic dependencies
- No split packages




```
java -p mods -m hello.world
```

Modular program structure



Guides for Java Modular Application

- http://openjdk.java.net/projects/jigsaw/quick-start

- A module is a set of packages designed for reuse.
- Modules offer strong encapsulation and reliable dependencies.
- Module-aware tooling is a big part of modular development.

Summary